B.GNANESHWAR
2303A51T01

LAB ASSIGNMENT - 13.1

Task Description #1 (Refactoring – Removing Code Duplication)

- Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.
- Instructions:
  - Prompt AI to identify duplicate logic and replace it with functions or classes.
  - Ensure the refactored code maintains the same output.
  - Add docstrings to all functions.

task1.py > ...

```python
def print_rectangle_details(length, width):
    """
    Calculate and print the area and perimeter of a rectangle.

    Parameters:
    length (int or float): The length of the rectangle.
    width (int or float): The width of the rectangle.

    Returns:
    None
    """
    area = length * width
    perimeter = 2 * (length + width)

    print("Area of Rectangle:", area)
    print("Perimeter of Rectangle:", perimeter)


# Rectangle 1
print_rectangle_details(5, 10)

# Rectangle 2
print_rectangle_details(7, 12)

# Rectangle 3
print_rectangle_details(10, 15)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
bandarignaneshwar@BANDARIs-MacBook-Air 10.3.py % /opt/homebrew/bin/python3 "/Users/bandarignaneshwar/Desktop
/ai coding/10.3.py/task1.py"
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
bandarignaneshwar@BANDARIs-MacBook-Air 10.3.py %
```

TASK 2 :

Task Description #2 (Refactoring – Extracting Reusable Functions)

- Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.
- Instructions:

task2.py > ...

```python
#TASK 2
#Refactoring – Extracting Reusable Functions
def calculate_total(price):
    """
    Calculate total price including 18% tax.

    Parameters:
    price (float): Base price

    Returns:
    float: Total price including tax
    """
    tax = price * 0.18
    return price + tax


prices = [250, 500]

for p in prices:
    print("Total Price:", calculate_total(p))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
/opt/homebrew/bin/python3 "/Users/bandarignaneshwar/Desktop/ai coding/10.3.py/task2.py"
bandarignaneshwar@BANDARIs-MacBook-Air 10.3.py % /opt/homebrew/bin/python3 "/Users/bandarignaneshwar/Desktop
/ai coding/10.3.py/task2.py"
Total Price: 295.0
Total Price: 590.0
bandarignaneshwar@BANDARIs-MacBook-Air 10.3.py %
```

TASK 3 :

#task 3
#Refactoring Using Classes and Methods
class GradeCalculator:

task3.py > ...

```python
#Refactoring Using Classes and Methods
class GradeCalculator:
    """
    A class to calculate student grades based on marks.
    """

    def calculate_grade(self, marks):
        """
        Determine grade based on marks.

        Parameters:
        marks (int): Student marks (0-100)

        Returns:
        str: Grade result
        """
        if 90 <= marks <= 100:
            return "Grade A"
        elif marks >= 80:
            return "Grade B"
        elif marks >= 70:
            return "Grade C"
        elif marks >= 40:
            return "Grade D"
        else:
            return "Fail"


calculator = GradeCalculator()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
/opt/homebrew/bin/python3 "/Users/bandarignaneshwar/Desktop/ai coding/10.3.py/task3.py"
bandarignaneshwar@BANDARIs-MacBook-Air 10.3.py % /opt/homebrew/bin/python3 "/Users/bandarignaneshwar/Desktop
/ai coding/10.3.py/task3.py"
Marks: 85 → Grade B
Marks: 72 → Grade C
Marks: 95 → Grade A
Marks: 35 → Fail
bandarignaneshwar@BANDARIs-MacBook-Air 10.3.py %
```

TASK 4 👍:

#TASK 4
#Refactoring – Converting Procedural Code to Functions
def calculate_area(radius):

task4.py > ...

```python
#Refactoring – Converting Procedural Code to Functions
def calculate_area(radius):
    """
    Calculate the area of a circle.

    Parameters:
    radius (float): Radius of the circle

    Returns:
    float: Area of the circle
    """
    import math
    return math.pi * radius ** 2
radii = [3, 5, 7]
for r in radii:
    print("Area of Circle:", calculate_area(r))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
/opt/homebrew/bin/python3 "/Users/bandarignaneshwar/Desktop/ai coding/10.3.py/task4.py"
bandarignaneshwar@BANDARIs-MacBook-Air 10.3.py % /opt/homebrew/bin/python3 "/Users/bandarignaneshwar/Desktop
/ai coding/10.3.py/task4.py"
Area of Circle: 28.274333882308138
Area of Circle: 78.53981633974483
Area of Circle: 153.93804002589985
bandarignaneshwar@BANDARIs-MacBook-Air 10.3.py %
```

TASK 5 :

#Refactoring Procedural Code into OOP Design
class Employeesalarycalculator:

task5.py > ...

```python
#TASK 5
#Refactoring Procedural Code into OOP Design
class Employeesalarycalculator:
    """
    A class to calculate employee salary based on hours worked and hourly rate.
    """

    def calculate_salary(self, hours_worked, hourly_rate):
        """
        Calculate total salary.

        Parameters:
        hours_worked (int): Number of hours worked
        hourly_rate (float): Hourly wage

        Returns:
        float: Total salary
        """
        return hours_worked * hourly_rate
salary_calculator = Employeesalarycalculator()
employee_data = [(40, 15), (35, 20), (45, 18)]
for hours, rate in employee_data:
    print("Total Salary:", salary_calculator.calculate_salary(hours, rate))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
/opt/homebrew/bin/python3 "/Users/bandarignaneshwar/Desktop/ai coding/10.3.py/task5.py"
bandarignaneshwar@BANDARIs-MacBook-Air 10.3.py % /opt/homebrew/bin/python3 "/Users/bandarignaneshwar/Desktop
/ai coding/10.3.py/task5.py"
Total Salary: 600
Total Salary: 700
Total Salary: 810
bandarignaneshwar@BANDARIs-MacBook-Air 10.3.py %
```

TASK 6 :

Task 6 (Optimizing Search Logic)

- **Task: Refactor inefficient linear searches using appropriate data structures.**

task6.py > ...

```python
def check_access(users_set, username):
    users_set (set): A set containing valid usernames.
    username (str): The username to search for.

    Returns:
    bool: True if user exists, False otherwise.
    """
    return username in users_set


# Using a set for O(1) lookup
users = {"admin", "guest", "editor", "viewer"}

name = input("Enter username: ")

if check_access(users, name):
    print("Access Granted")
else:
    print("Access Denied")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
/opt/homebrew/bin/python3 "/Users/bandarignaneshwar/Desktop/ai coding/10.3.py/task6.py"
bandarignaneshwar@BANDARIs-MacBook-Air 10.3.py % /opt/homebrew/bin/python3 "/Users/bandarignaneshwar/Desktop
/ai coding/10.3.py/task6.py"
Enter username: admin
Access Granted
bandarignaneshwar@BANDARIs-MacBook-Air 10.3.py %
```

TASK 7 :

Task 7 – Refactoring the Library Management System
Problem Statement

task7.py > ...

```python
else:
    print("Book already exists.")

# Adding second book (duplicate logic)
title = "AI Fundamentals"
author = "Jane Smith"
isbn = "102"

if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")

# Searching book (repeated logic structure)
isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")

# Removing book (again repeated pattern)
isbn = "101"
if isbn in library_db:
    del library_db[isbn]
    print("Book removed successfully.")
else:
    print("Book not found.")

# Searching again
isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
bandarignaneshwar@BANDARIs-MacBook-Air 10.3.py % /opt/homebrew/bin/python3 "/Users/bandarignaneshwar/Desktop
/ai coding/10.3.py/task7.py"
Book added successfully.
Book added successfully.
Book Found: {'title': 'Python Basics', 'author': 'John Doe'}
Book removed successfully.
Book not found.
bandarignaneshwar@BANDARIs-MacBook-Air 10.3.py %
```

TASK 8 :

Task 8– Fibonacci Generator

Write a program to generate Fibonacci series up to n.

The initial code has:

- Global variables.
- Inefficient loop.
- No functions or modularity

task8.py > ...

```python
#task 8
#Fibonacci Generator
def fibonacci(n):
    """
    Generate Fibonacci sequence up to n terms.

    Parameters:
    n (int): Number of terms to generate

    Returns:
    list: Fibonacci sequence up to n terms
    """
    sequence = []
    a, b = 0, 1
    for _ in range(n):
        sequence.append(a)
        a, b = b, a + b
    return sequence
num_terms = 10
print("Fibonacci Sequence:", fibonacci(num_terms))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
/opt/homebrew/bin/python3 "/Users/bandarignaneshwar/Desktop/ai coding/10.3.py/task8.py"
bandarignaneshwar@BANDARIs-MacBook-Air 10.3.py % /opt/homebrew/bin/python3 "/Users/bandarignaneshwar/Desktop
/ai coding/10.3.py/task8.py"
Fibonacci Sequence: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
bandarignaneshwar@BANDARIs-MacBook-Air 10.3.py %
```

TASK 9 :

Task 9 – Twin Primes Checker

task9.py > ...

```python
#task 9
#Twin Primes Checker
def is_prime(num):
    """
    Check if a number is prime.

    Parameters:
    num (int): Number to check

    Returns:
    bool: True if prime, False otherwise
    """
    if num <= 1:
        return False
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            return False
    return True
def twin_primes(n):
    """
    Generate twin primes up to n.
    Parameters:
    n (int): Upper limit for twin primes
    Returns:
    list: List of twin prime pairs up to n
    """
    twins = []
    for i in range(2, n - 1):
        if is_prime(i) and is_prime(i + 2):
            twins.append((i, i + 2))
    return twins
upper_limit = 100
print("Twin Primes up to", upper_limit, ":", twin_primes(upper_limit))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
/opt/homebrew/bin/python3 "/Users/bandarignaneshwar/Desktop/ai coding/10.3.py/task9.py"
bandarignaneshwar@BANDARIs-MacBook-Air 10.3.py % /opt/homebrew/bin/python3 "/Users/bandarignaneshwar/Deskt
/ai coding/10.3.py/task9.py"
Twin Primes up to 100 : [(3, 5), (5, 7), (11, 13), (17, 19), (29, 31), (41, 43), (59, 61), (71, 73)]
bandarignaneshwar@BANDARIs-MacBook-Air 10.3.py %
```

TASK 10 :

Task 10 – Refactoring the Chinese Zodiac Program

task10.py > ...

```python
#task 10
#Refactoring the Chinese Zodiac Program
def get_zodiac(year):
    """
    Return Chinese zodiac sign for a given year.
    """
    zodiac = [
        "Monkey", "Rooster", "Dog", "Pig",
        "Rat", "Ox", "Tiger", "Rabbit",
        "Dragon", "Snake", "Horse", "Goat"
    ]
    return zodiac[year % 12]


year = int(input("Enter a year: "))
print(get_zodiac(year))
```

Ports

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
/opt/homebrew/bin/python3 "/Users/bandarignaneshwar/Desktop/ai coding/10.3.py/task10.py"
bandarignaneshwar@BANDARIs-MacBook-Air 10.3.py % /opt/homebrew/bin/python3 "/Users/bandarignaneshwar/Desktop
/ai coding/10.3.py/task10.py"
Enter a year: 2005
Rooster
bandarignaneshwar@BANDARIs-MacBook-Air 10.3.py %
```